

FIȘĂ DE LUCRU

Metoda de sortare prin selecția minimului

Clasa a IX-a – Informatică (CS) – Matematica-Informatică

1. Noțiuni teoretice

Sortarea este procesul de aranjare a elementelor unei liste într-o anumită ordine (crescătoare sau descrescătoare).

Principiul metodei selecției minimului:

Se parcurge lista de n ori. La fiecare parcurgere i (de la 0 la $n-2$):

1. Se caută elementul minim din sublista nesortată (de la poziția i până la sfârșit)
2. Se interschimbă elementul minim găsit cu elementul de pe poziția i
3. După această parcurgere, elementul de pe poziția i este cel corect (cel mai mic din sublista rămasă)

Complexitate: $O(n^2)$ comparații în toate cazurile (favorabil, mediu, nefavorabil)

Număr de interschimbări: maxim $n-1$

2. Exemplu pas cu pas

Fie lista: [64, 25, 12, 22, 11]. Vrem să o sortăm crescător.

Pasul 1 ($i=0$): Minimul din [64, 25, 12, 22, 11] este 11 (poziția 4). Se interschimbă cu elementul de pe poziția 0.

Rezultat: [11, 25, 12, 22, 64]

Pasul 2 ($i=1$): Minimul din [25, 12, 22, 64] este 12 (poziția 2). Se interschimbă cu elementul de pe poziția 1.

Rezultat: [11, 12, 25, 22, 64]

Pasul 3 ($i=2$): Minimul din [25, 22, 64] este 22 (poziția 3). Se interschimbă cu elementul de pe poziția 2.

Rezultat: [11, 12, 22, 25, 64]

Pasul 4 ($i=3$): Minimul din [25, 64] este 25 (poziția 3). Este deja pe poziția corectă.

Rezultat final: [11, 12, 22, 25, 64]

3. Implementare în Python

```
def sortare_selectie(lista):
    n = len(lista)
    for i in range(n - 1):
        # Cautăm poziția minimului din sublista [i..n-1]
        poz_min = i
        for j in range(i + 1, n):
            if lista[j] < lista[poz_min]:
                poz_min = j
        # Interschimbăm elementul minim cu cel de pe poziția i
```

```
        lista[i], lista[poz_min] = lista[poz_min], lista[i]

# Exemplu de utilizare:
numere = [64, 25, 12, 22, 11]
print('Inainte de sortare:', numere)
sortare_selectie(numere)
print('Dupa sortare:', numere)
```

Observație: Funcția modifică lista primită ca parametru (listele sunt mutabile în Python).

4. Variantă cu afișare pas cu pas (pentru înțelegere)

```
def sortare_selectie_vizual(lista):
    n = len(lista)
    for i in range(n - 1):
        poz_min = i
        for j in range(i + 1, n):
            if lista[j] < lista[poz_min]:
                poz_min = j
        if poz_min != i:
            print(f'Pasul {i+1}: interschimb {lista[i]} cu {lista[poz_min]}')
            lista[i], lista[poz_min] = lista[poz_min], lista[i]
        else:
            print(f'Pasul {i+1}: elementul {lista[i]} este deja pe pozitia corecta')
    print(f'  Lista: {lista}')
    print(f'Rezultat final: {lista}')
```

5. Exerciții

Exercițiul 1 (ușor)

Aplicați manual metoda selecției minimului pentru lista [38, 27, 43, 3, 9, 82, 10]. Scrieți rezultatul după fiecare pas.

Exercițiul 2 (ușor)

Câte comparații se efectuează pentru sortarea unei liste de 6 elemente prin selecția minimului? Calculați folosind formula $n*(n-1)/2$.

Exercițiul 3 (mediu)

Modificați funcția de sortare astfel încât să sorteze descrescător (prin selecția maximului).

Exercițiul 4 (mediu)

Se citesc de la tastatură n note (numere întregi între 1 și 10). Scrieți un program care sortează notele crescător prin selecția minimului și afișează lista sortată.

Exercițiul 5 (avansat)

Se dau 200 de punctaje (numere întregi între 0 și 100) obținute de participanții la olimpiada de informatică. Scrieți un program care determină și afișează primele 10 cele mai mari punctaje, folosind selecția minimului adaptată (hint: după 10 pași de selecție a maximului, aveți primele 10 valori sortate).

Exercițiul 6 (avansat)

Comparați experimental numărul de comparații și interschimbări efectuate de metoda selecției minimului pentru: a) o listă deja sortată crescător; b) o listă sortată descrescător; c) o listă aleatoare. Modificați funcția de sortare pentru a număra operațiile.

6. Rezumat

- Selecția minimului găsește, la fiecare pas, cel mai mic element din porțiunea nesortată și îl plasează pe poziția corectă.
- Complexitatea este $O(n^2)$ în toate cazurile (favorabil, mediu, nefavorabil).
- Numărul maxim de interschimbări este $n-1$, ceea ce o face eficientă când interschimbările sunt costisitoare.
- Metoda este simplă de înțeles și de implementat, dar nu este eficientă pentru liste foarte mari.